A Stochastic Process Approach to Tennis Match Simulation

Sebastian Villanueva
Final Project - MATH 5652

Spring 2026

Abstract

This project is a simulation-based framework for modeling a tennis match using stochastic processes. Tennis scoring can be modeled with probability at different hierarchical levels: points, games, sets, and matches. We construct a discrete-time stochastic process that evolves through these levels, incorporating player-specific probabilities for serving and returning. The model is implemented using Monte Carlo simulation to estimate match outcome probabilities. We analyze the structure of the process, interpret it as a system of Markov chains, and discuss the implications of assumptions such as independence and stationarity.

1 Introduction

Tennis is a sport with an inherently probabilistic structure. Each point can be viewed as a random experiment whose outcome depends on the players' abilities. From these point-level outcomes, higher-level structures such as games, sets, and matches emerge through deterministic scoring rules.

This project models a tennis match as a stochastic process evolving over discrete states. The primary goal is to estimate match-winning probabilities for different players using simulation techniques. By framing the problem within stochastic process theory, we connect practical simulation methods to theoretical constructs such as Markov chains and renewal processes.

2 Modeling Framework

2.1 Hierarchical Structure of the Process

The tennis match is modeled as a hierarchical stochastic system with four levels:

- **Point level:** The fundamental random event.
- **Game level:** A sequence of points until a player wins by at least 2 points after reaching 4.
- **Set level:** A sequence of games until a player wins at least 6 games with a 2-game margin.
- **Match level:** A sequence of sets until a player wins a majority (best-of format).

Each level aggregates outcomes from the previous level, forming a nested stochastic process.

2.2 Point-Level Probability Model

Let Player A and Player B have:

- $p_{A_{serve}}$ = probability Player A wins a point on serve
- $p_{A_{return}}$ = probability Player A wins a point on return

When Player A serves, the probability that A wins the point is modeled as:

$$p_{A_{win}} = \frac{p_{A_{serve}} + (1 - p_{B_{return}})}{2}$$

Similarly, when Player B serves:

$$p_{A_{win}} = \frac{(1 - p_{B_{serve}}) + p_{A_{return}}}{2}$$

This symmetric formulation incorporates both server strength and return weakness, creating a balanced probabilistic model.

2.3 Discrete-Time Stochastic Process Representation

Let X_n denote the state of the match after the n-th point. The state includes:

- Current point score
- Current game score
- Current set score
- Server identity

The sequence $\{X_n\}_{n \geq 0}$ forms a discrete-time stochastic process.

3 Markov Property

The system satisfies the **Markov property** under the modeling assumptions:

$$P(X_{n+1} \mid X_n, X_{n-1}, \dots) = P(X_{n+1} \mid X_n)$$

This holds because:

- Point outcomes are independent given the current server
- Future evolution depends only on the current score and server

Thus, the tennis match can be viewed as a **finite-state Markov chain** with absorbing states corresponding to match completion.

3.1 State Space

The state space is large but finite, consisting of all possible:

- Point scores (including deuce/advantage)
- Game scores within a set
- Set scores within a match

Absorbing states occur when a player wins the match.

4 Simulation Methodology

4.1 Monte Carlo Simulation

Because the state space is complex, analytical computation of match probabilities is difficult. Instead, we use Monte Carlo simulation.

We simulate N independent matches and estimate:

$$P(\text{Player A wins}) \approx \frac{\text{Number of wins by A}}{N}$$

By the Law of Large Numbers, this estimator converges to the true probability as $N \to \infty$.

4.2 Algorithm Structure

Each simulation proceeds as follows:

1. Randomly select the initial server
2. Simulate points until a game is won
3. Simulate games until a set is won
4. Simulate sets until the match is won

The independence between simulations ensures unbiased estimation.

5 Implementation Details

The model is implemented in Python using object-oriented design:

- **Player class:** Stores serving and returning probabilities
- **TennisMatch class:** Handles simulation logic

Key components include:

- `simulate_point`: Bernoulli trial based on server
- `simulate_game`: Sequential process with stopping condition
- `simulate_set`: Incorporates game structure and tiebreak
- `monte_carlo`: Repeats full match simulation

The simulation uses pseudo-random number generation to approximate the underlying probability distributions.

6 Results and Interpretation

Using realistic player parameters, we simulate match outcomes between professional players. For example:

- Reilly Opelka: Strong server:
  Pr(Winning on serve) = 0.7, Pr(Winning on return) = 0.28
- Alex De Minaur: Strong returner:
  Pr(Winning on serve) = 0.64, Pr(Winning on return) = 0.40
- After a simulation of 10000 matches:
  Reilly Opelka win probability: 0.3303
  Alex De Minaur win probability: 0.6697

Similarly,

- Carlos Alcaraz: Strong all-court player:
  Pr(Winning on serve) = 0.67, Pr(Winning on return) = 0.42
- Novak Djokovic: Veteran all-court player:
  Pr(Winning on serve) = 0.68, Pr(Winning on return) = 0.42
- After a simulation of 10000 matches:
  Carlos Alcaraz win probability: 0.4669
  Novak Djokovic win probability: 0.5331

The results show that:

- Small differences in point probabilities lead to significant differences in match outcomes. (e.g The difference in serve win percentage between Alcaraz and Djokovic is a mere 1% but appears to have a notable effect on the overall win probability)
- Serving advantage propagates through all levels of the match
- Variability remains high due to the stochastic nature of the process

This highlights how randomness at the micro-level (points) amplifies through the hierarchical structure.

7 Discussion

7.1 Model Assumptions

The model relies on several simplifying assumptions:

- Independence of points
- Constant probabilities (stationarity)
- No psychological or fatigue effects

In reality, these assumptions may not hold. For example, momentum effects could introduce dependence between points.

7.2 Extensions

The model can be extended in several ways:

- Non-homogeneous Markov chains (time-varying probabilities)
- Incorporating fatigue or pressure effects
- Incorporating different win probabilities based on court material
- Bayesian updating of player probabilities

These extensions would make the model more realistic but also more complex.

8 Conclusion

This project demonstrates how a real-world system like a tennis match can be modeled using stochastic processes. By viewing the match as a system of Markov chains and applying Monte Carlo simulation, we obtain meaningful probabilistic insights into match outcomes.

The framework illustrates key concepts from stochastic processes, including:

- Markov property
- Absorbing states
- Law of Large Numbers

Overall, the project highlights the power of stochastic modeling in capturing complex, multi-level systems driven by randomness.

Appendix: Code

GitHub Link: `https://github.com/sebvillanueva/5652FinalProject_SebastianVillanueva.git`

References:
Official ATP Website: `https://www.atptour.com/en`

Use of AI: ChatGPT was used to point out syntax errors in my code as well as help format the outputs in a way that was clean.